

COMP 3300 – Fall 2025
Assignment 6, 10 pts.
Due: Sunday, Oct 19th @
11:59pm

Objectives

- Create a Simple GUI application that allows for reading/writing files.

Notes

- Please read through this entire document before you begin, so you know all the requirements for the program at the outset.
- **Any program that does not compile will receive a zero (0). Partial credit is not possible for any program that does not compile.**

Getting Started

1. Before you start make sure you have reviewed the slides for the week and read up on the provided links to the MS documentation.
2. There are no external libraries permitted. You must accomplish this using only the native C#/.Net provided elements.
3. This assignment is very small in scale. The focus is mainly on reading in data, transforming it to C# models, then writing the data back out. The GUI portion is simply there to allow ease of use when running these behaviors.
4. Start Visual Studio 2022 and create a new C# Windows Forms App (**not** .Net Framework).
 - a. Name the project *FirstnameLastnameA6*, e.g., SallieMaeA6.
5. Change the form title bar to be *Firstname Lastname Assignment 6*.
6. The default form should be named appropriately (based on what it is). Do not use the default Form1 name given by Visual Studio.

Requirements

For all GUI based portions, you are free to design/layout the screen as you would like provided the requested buttons are present on the window. You are not required to implement things like tab order and such for this assignment. The GUI items are listed as # 1 and # 2 below but you may want to go ahead and get the Model elements in order before starting on the GUI.

1. Add 3 buttons to the form. Each should perform one the following actions:
 - a. Choose File:
 - i. This button should open a window that allows the user to select a specified file located on their computer. You can assume the user will be able to locate and load the correct file. You are provided the file that is to be read in.
 - ii. Once the file is loaded, the file will be “read” in and converted to an appropriate collection of C# model objects.
 - iii. After loading the data and converting to the model objects, that collection of objects should be stored within the form class in an appropriate property.
 - b. Export Data as CSV:
 - i. This button should take the loaded model data from Choose File and export it back to a CSV file.
 1. Each “row” of the CSV will represent a single “model” object with the property values for each object separated by a comma, as a CSV.
 2. In the source file, there are twenty-five (25) different objects represented. So, your output should have 25 rows.
 3. If there are no mapped objects present, this button should not function.
 - c. Export Data as JSON:
 - i. This button should take the loaded model data from Choose File and export it back to a JSON file with a different name.
 1. This is not simply copying the existing JSON file, but you should be “writing” from the models that are mapped in. If the original file is copied directly, no credit will be given. The goal is to write to a JSON data file from a source of C# model(s).
 2. If there are no mapped objects present, this button should not function.
2. Add a way to display the total count of “mapped” objects present within the collection read in. You may choose how to display this count, but it should always read the total count, even when there are no mapped in objects. And it should be clear what the number is (so it’s user friendly).
 - a. For example – starting the program there will be no objects mapped in, so the total count should read 0.
 - b. After mapping in the provided file, there should be 25 objects mapped in, so the count would read 25.

3. Create, in an appropriate namespace, C# Model that will represent a single object from the provided JSON data.
 - a. You should examine the JSON data from a text reader (or something like VS Code) to determine the structure of the object.
 - i. JSON objects begin with { and end with }
 - ii. Multiple objects are contained within an array, []
 - iii. Each object is separated by a comma.
 - b. The datatypes are not provided as they can be inferred from reading the JSON object data. Pay attention to identifiers like “ “ and decimal points as they can help you identify what data type to use.
 - c. The “Model” should have the appropriate properties to match the JSON object.
 - d. How you choose to structure the other parts of the Model is up to you, based on your research on mapping in data.

This is not a long assignment, but you will need to research a bit. There are several ways to get to mapping in JSON objects as C# models.

Grading breakdown

- 10 pts. – Exercise completed successfully with at most only a minor error and follows best programming practices.
- 7 pts. – Exercise is missing some functionality or has some errors in it and mostly follows best programming practices.
- 4 pts. – Exercise is missing most of the major functionality or has major errors or best programming practices not followed.
- 0 pts. – Exercise not submitted, does not compile, or barely started.

Submission

Make sure you clean your solution (Build → Clean Solution) and then zip up your project which needs to include the solution file into a ZIP file called *YourNameA6.zip* and submit the assignment by the due date.